

RAG Document Assistant

Project Report

A local, web-based document question-answering application built with HTML, JavaScript, Python, and FastAPI.

1. Executive Summary

This project implements a basic Retrieval-Augmented Generation (RAG)-style document assistant. Users ask questions through a browser chat interface. A FastAPI backend reads a local knowledge base, retrieves the most relevant paragraph using keyword overlap, and returns that evidence to the user.

The application demonstrates the full frontend-to-backend AI application flow: user input, API communication, document retrieval, answer delivery, error handling, and a responsive chat experience.

2. Problem Statement

Teams often spend time searching policy documents, procedures, and operational notes for simple answers. This project provides a lightweight assistant that can retrieve relevant information from a maintained knowledge base.

3. Objectives

- Build a browser-based chat interface for questions about internal documents.
- Create a FastAPI endpoint to receive and process questions.
- Retrieve the most relevant section from a local knowledge base.
- Return a clear fallback response when no relevant information is found.
- Add practical UI features such as Enter-to-send, a loading state, scrolling, and error handling.

4. Technology Stack

- Frontend: HTML, CSS, and vanilla JavaScript.
- Backend: Python, FastAPI, Uvicorn, and Pydantic.
- Knowledge base: Plain-text file (knowledge.txt).
- Development environment: VS Code and Live Server.

5. System Architecture

User -> index.html -> POST /ask -> FastAPI app.py -> knowledge.txt -> relevant paragraph -> JSON response -> browser chat.

6. Implementation Details

6.1 Frontend - index.html

The page contains a question input, Send button, and chat box. The sendMessage function takes the question, displays it, and uses fetch to send a JSON POST request to `http://127.0.0.1:8000/ask`. The returned answer is shown in the chat.

The addMessage helper creates DOM elements using `textContent` and `createTextNode` rather than inserting user text as HTML. This is safer because typed text is displayed as text, not executable markup.

- Enter key support sends the question without clicking Send.
- Thinking status gives the user feedback while a request is running.
- The input and button are disabled during the request to prevent duplicates.
- Automatic scrolling keeps the newest message visible.
- Error handling reports when the local backend is unavailable.

6.2 Backend - app.py

FastAPI exposes a POST endpoint named `/ask`. Pydantic validates that every request contains a question string. CORS middleware allows the browser page and local backend to communicate while they run on different ports.

The backend reads `knowledge.txt` and splits it into sections at blank lines. Each paragraph is therefore treated as one retrieval chunk.

6.3 Retrieval Logic

The words function converts text to lowercase and extracts alphabetic words of three or more letters. For every paragraph, the application counts the words shared with the user's question. The paragraph with the highest count is selected.

For example, a question containing 'maintenance' matches the paragraph 'System maintenance occurs every Sunday at midnight UTC.' If no words overlap, the app returns: I could not find an answer in the knowledge base.

7. RAG Explanation

RAG stands for Retrieval-Augmented Generation. A production RAG system retrieves relevant source text and passes it to a language model, which then writes a grounded answer. This project implements the retrieval foundation and returns the retrieved evidence directly. It does not yet use embeddings, a vector database, or an LLM.

8. How to Run the Project

- Install dependencies: `python -m pip install fastapi uvicorn`
- Start the backend: `python -m uvicorn app:app --reload`

- Open index.html with Live Server.
- Ask a question that contains terms found in knowledge.txt.

9. Current Limitations

- Retrieval depends on exact keyword overlap; it does not understand semantic similarity.
- Only plain-text knowledge is supported.
- The system returns source text rather than an LLM-generated answer.
- The local CORS configuration allows all origins and should be restricted before deployment.

10. Future Enhancements

- Use embeddings and ChromaDB or FAISS for semantic search.
- Support PDF, CSV, and document uploads.
- Use an LLM to generate concise answers only from retrieved context.
- Add source citations, confidence scores, authentication, and a SQLite audit log.
- Extend the solution into an operations exception-resolution copilot for policies, invoices, and SOPs.

11. Conclusion

The RAG Document Assistant successfully proves an end-to-end AI application workflow. It provides a strong foundation for a more advanced enterprise knowledge assistant because its components are modular: the interface, API, retrieval logic, and knowledge base can each be upgraded independently.